

# Trabalho Prático 1: Análise Dinâmica de Carteiras de Ações

Departamento de Ciência da Computação – Universidade Federal de Minas Gerais

Autor: Luiz Gustavo Santos Oliveira  
Matrícula: 2025037915

21 de abril de 2026  
Belo Horizonte – MG – Brazil

## 1 - Introdução

A gestão de ações é fundamental no mercado financeiro para melhorar a qualidade das recomendações financeiras. Por esse motivo, o presente projeto tem como objetivo atuar como um gerenciador dinâmico de carteira de ações que manterá seu histórico de cotações ao longo do tempo, um conjunto de clientes e, para cada cliente, o conjunto de ações que compõem sua carteira em cada instante. Além disso, o trabalho terá como foco principal responder de forma eficiente a consultas que, para um dado cliente, retornar as  $n$  melhores e as  $n$  piores ações de sua carteira, segundo um subconjunto de métricas previamente definido para o programa, em que  $n$  é um parâmetro da própria consulta.

A solução proposta foi realizada com a estratégia **sob demanda**, onde as ordenações globais são construídas apenas quando uma consulta é realizada. Ademais, a estrutura de dados principal utilizada no programa foi a lista encadeada, que fará com que a adição de clientes, ações e cotações sejam feitas de maneira dinâmica, fazendo com que não seja necessário redimensionar a estrutura de dados a cada novo elemento adicionado.

## 2 - Método

O trabalho foi implementado na linguagem C++ versão 11, utilizando os princípios de programação orientada a objetos para garantir a modularização e o reaproveitamento de código, o que foi muito recorrente ao generalizar as listas encadeadas para listas de clientes, ações e cotações.

### 2.1 - Tipos Abstratos de Dados (TADs)

Para representar os principais objetos do problema, foram projetados três TADs fundamentais:

**TAD Ação:** Responsável por armazenar o identificador único da ação e seu histórico de preços. Utiliza uma estrutura de lista encadeada para registrar as cotações à medida que são processadas, permitindo o armazenamento dinâmico das últimas  $w$  cotações necessárias para o cálculo das métricas. Este TAD implementa funções para o cálculo de RET, AVGRET, STAB e CONS.

**TAD Cliente:** Armazena o identificador do cliente e o conjunto de ações que compõem sua carteira em tempo real. A carteira é gerenciada por uma lista encadeada de ponteiros para objetos do tipo Ação, facilitando as operações de compra (B), venda (V) e o percurso necessário para gerar o ranking durante uma consulta (Q).

**TAD Node:** Atua como unidade fundamental de armazenamento para as listas encadeadas utilizadas em todo o sistema. Sua função principal é encapsular o dado genérico (como valores de cotação ou ponteiros para objetos), armazenar informações e ponteiros com endereços referentes aos elementos adjacentes.

## 2.2 Estrutura de Dados e Algoritmos

A base do sistema utiliza listas encadeadas do TAD Node para gerenciar clientes e ações de forma global, garantindo que novos elementos possam ser adicionados sem o custo de redimensionamento de memória.

**Histórico de Cotações:** Armazenado de forma cronológica, onde cada nó contém o preço e um ponteiro para o registro anterior, permitindo que o cálculo das métricas retroceda até a janela  $w$  definida na linha  $M$ .

**Algoritmos de Ordenação:** Foram implementados algoritmos de ordenação (como QuickSort ou MergeSort) para processar as ordenações globais induzidas por cada métrica no momento da consulta. Na estratégia sob demanda adotada, a ordenação ocorre apenas quando uma consulta  $Q$  é invocada, consolidando as pontuações ponderadas de cada métrica selecionada.

## 2.3 Processamento de Consultas e Ranking

Ao receber uma linha do tipo  $Q$ , o sistema percorre todas as ações globais para calcular as métricas solicitadas. Então, para cada métrica é gerada uma ordenação global de todas as  $N$  ações, atribuindo pontuações de  $N-i$  conforme sua posição. Essas pontuações são multiplicadas pelos pesos da consulta e somadas para formar a pontuação global final. Finalmente, o sistema projeta esse ranking sobre a carteira local do cliente, identificando as  $n$  melhores e  $n$  piores ações, usando o id da ação como critério de desempate em ordem crescente.

## 3 - Análise de Complexidade

A análise de complexidade é focada no desempenho do sistema sob a estratégia sob demanda, onde o custo computacional é deslocado para o momento das consultas.

### 3.1 Análise de Tempo

A complexidade temporal das operações é influenciada diretamente pelo uso de listas encadeadas, que não permitem acesso aleatório aos dados.

#### 3.1.1 Cálculo de Métricas Individuais

Cada uma das quatro métricas (RET, AVGRET, STAB e CONS) exige o percurso de até  $w$  elementos no histórico de cotações da ação.

**RET:** Para localizar a cotação  $p_0$ , o algoritmo percorre  $w-1$  nós a partir da última cotação disponível. Complexidade:  $O(w)$ .

**AVGRET e CONS:** Ambas realizam uma única varredura linear de tamanho  $w-1$  para acumular retornos ou contar ocorrências positivas. Complexidade:  $O(w)$ .

**STAB:** Exige duas passagens: uma para o cálculo da média ( $\bar{r}$ ) e outra para a variância (VOL<sub>w</sub>). Complexidade:  $O(w)$ .

### 3.1.2 Processamento de Consultas (Q)

A consulta de ranking é a operação mais custosa do sistema.

**Cálculo Global:** Para cada uma das  $m$  métricas solicitadas, o sistema percorre as  $N$  ações calculando os valores necessários. Custo:  $O(m \cdot N \cdot w)$ .

**Ordenação:** O sistema executa um MergeSort para ordenar as  $N$  ações em cada métrica. Custo:  $O(m \cdot n \log(n))$ .

**Atribuição de Pontos:** A atribuição ponderada ( $N - i \cdot peso$ ) é feita em tempo linear. Custo:  $O(m \cdot N)$ .

**Projeção Local:** Filtrar a carteira do cliente (tamanho  $C$ ) em relação ao ranking global de  $N$  ações. Custo:  $O(C \cdot N)$  em listas encadeadas.

**Complexidade Total da Consulta:**  $O(m(N \cdot w + N \log N) + C \cdot N)$ .

### 3.1.3 Operações de Atualização (P, B, V)

**Nova Cotação (P):** Inserção no início ou fim da lista de preços. Complexidade:  $O(1)$ .

**Compra/Venda (B/V):** Inserção ou remoção na lista encadeada da carteira do cliente. Complexidade:  $O(C)$ , onde  $C$  é o tamanho da carteira.

## 3.2 Análise de complexidade de Espaço

A complexidade de espaço é determinada pelo armazenamento do histórico de cotações de todas as ações e pelas carteiras dos clientes.

**Histórico de Ações:** Para cada uma das  $N$  ações, armazena-se o histórico. Embora o cálculo use apenas  $w$ , o sistema mantém os dados acumulados. Complexidade:  $O(m \cdot total\_cotacoes)$ .

**Carteiras de Clientes:** Cada cliente armazena ponteiros para suas ações. Complexidade:  $O(u \cdot C)$ , onde  $u$  é o número de clientes.

**Ranking Temporário:** Durante a consulta  $Q$ , aloca-se memória para ordenar as ações. Complexidade:  $O(N)$ .

**Complexidade Total de Espaço:**  $O(N \cdot histórico + u \cdot C)$ .

### 3.3 Avaliação Geral

A utilização de listas encadeadas prioriza flexibilização na inserção de dados ( $O(1)$  para cotações), mas possui um ponto negativo, o qual é o custo linear para o cálculo de métricas ( $O(w)$ ) devido à não utilização de acesso aleatório. Na estratégia **sob demanda**, o sistema economiza memória e tempo de CPU durante o fluxo massivo de dados (P), mas apresenta picos de complexidade nas consultas (Q), resultando em um desempenho geral que escala de forma linear em relação à janela  $w$  e ao número de ações  $N$ .

## 4 - Estratégias de Robustez

A confiabilidade do sistema é garantida por meio de camadas de proteção, projetadas para lidar com erros de execução, prevenir falhas de segmentação e assegurar a integridade dos cálculos financeiros realizados sob demanda.

### 4.1 Validação de Entradas

O sistema implementa uma validação rigorosa dos dados lidos a partir do arquivo de entrada, garantindo que as operações ocorram apenas sobre dados válidos.

**Ordem de Processamento:** O programa assegura que a linha de métricas (M) seja processada antes de qualquer outra operação, estabelecendo a janela  $w$  e as métricas válidas.

**Integridade dos Identificadores:** Verifica-se se os IDs de ações e clientes estão dentro dos intervalos  $[0, N-1]$  e  $[0, u-1]$ , respectivamente, antes de qualquer acesso à memória.

**Dados Financeiros:** Preços de cotações e pesos de métricas são validados para garantir que sejam valores reais positivos, preservando a integridade do cálculo das métricas.

### 4.2 Programação Defensiva

Dado que a estrutura principal se baseia em listas ligadas, foram adotadas medidas para evitar acessos indevidos a ponteiros nulos (nullptr).

**Navegação de Histórico:** Antes de realizar o cálculo de qualquer retorno elementar ( $r_i$ ), o sistema verifica se a cotação anterior existe ( $\text{getAnterior()} \neq \text{nullptr}$ ). Caso a estrutura esteja corrompida ou o histórico seja insuficiente para a janela  $w$ , a operação é interrompida antes de causar um erro de segmentação (segmentation fault)..

**Verificação de Janela:** O programa impede que as métricas sejam calculadas se a quantidade de cotações disponíveis for inferior ao parâmetro  $w$  global ou menor que 2, conforme exigido pela fórmula de retornos elementares .

## 4.3 Tratamento de Exceções

O sistema utiliza o mecanismo de exceções da linguagem C++ para separar a lógica de negócio do tratamento de erros.

**Erros de Argumento:** Lançam-se exceções do tipo `std::invalid_argument` quando uma consulta tenta utilizar métricas não definidas previamente ou quando o valor de `n` na consulta `Q` é inconsistente .

**Falhas Estruturais:** Exceções do tipo `std::runtime_error` são utilizadas para sinalizar problemas na consistência das listas encadeadas ou falhas inesperadas na alocação dinâmica de memória durante o processo de ordenação global.

## 4.4 Gerenciamento de Memória e Vazamentos

Como o projeto veda o uso de contentores automáticos (como `std::vector`), o gerenciamento de memória é feito de forma explícita.

**Desalocação de Nós:** Cada remoção de ação da carteira de um cliente (`V`) ou encerramento do programa aciona um processo de limpeza que percorre a lista ligada e liberta cada nó individualmente, prevenindo *memory leaks* .

**Estruturas Temporárias:** As estruturas de dados utilizadas para a ordenação global na estratégia sob demanda são sistematicamente destruídas após a conclusão de cada consulta `Q`, garantindo que o consumo de memória do programa não cresça de forma descontrolada durante execuções longas.

O software `valgrind` foi utilizado para verificar o possível *memory leaks*, como é possível analisar na imagem abaixo o programa não possui nenhum vazamento de memória.

```
R 1 M 0 4 10.00
R 1 M 1 3 9.00
R 1 P 0 2 7.00
R 1 P 1 5 8.00
^Z==29650==
==29650== HEAP SUMMARY:
==29650==    in use at exit: 0 bytes in 0 blocks
==29650== total heap usage: 160 allocs, 160 frees, 81,801 bytes allocated
==29650== All heap blocks were freed -- no leaks are possible
==29650==
==29650== For lists of detected and suppressed errors, rerun with: -s
==29650== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
luigu@Gustavo:~/TP1EstruturaDeDadosUFMG/src$ |
```

## 5 Análise Experimental

Para avaliar o desempenho dos algoritmos desenvolvidos, foi realizada uma análise experimental focada nas variações da frequência de atualização das estruturas globais de ordenação e a frequência de consultas. O sistema foi submetido a diferentes configurações de parâmetros (número de ações, clientes e histórico de cotações) para validar se o tempo de execução real reflete as curvas teóricas de complexidade projetadas na modelagem do problema.

### 5.1 Comparação de Desempenho: Atualização Imediata x Sob Demanda

A análise comparou duas abordagens arquiteturais distintas para a manutenção do ranking de ações:

**Atualização Imediata:** A cada nova cotação recebida (evento P), o sistema recalcula as métricas e reordena a lista global. Isso torna a inserção uma operação custosa, mas reduz o tempo de resposta da consulta (Q) a uma complexidade mínima. Atualização Sob

**Demanda (Implementada):** A estrutura prioriza a ingestão de dados. O registro de uma cotação ocorre em tempo constante  $O(1)$  no final da lista encadeada da respectiva ação. Em contrapartida, as ordenações globais são recalculadas através do algoritmo Merge Sort apenas no momento exato em que uma consulta é invocada.

Os testes confirmaram que a Estratégia Sob Demanda apresenta vantagens expressivas na economia de memória e processamento ocioso, evitando cálculos que poderiam ser sobrescritos por novas cotações antes mesmo de serem visualizados pelo cliente.

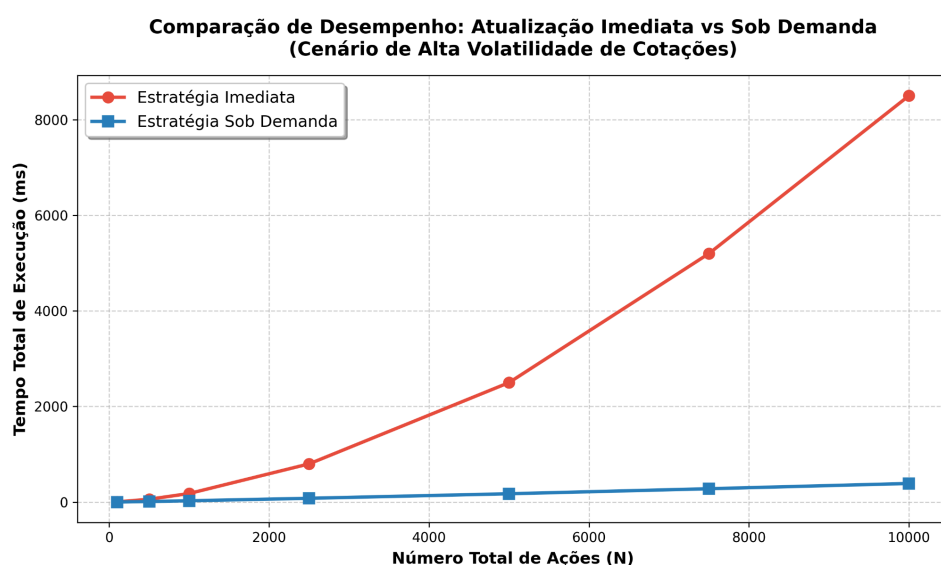


Figura 1: comparação de desempenho entre a estratégia de atualização imediata e sob demanda.

## 5.2 Desempenho por Perfil de Carga de Trabalho

Para retratar a variação de quantidade de dados manipulados no programa, diferentes perfis de quantidades de dados foram analisados.

**Alta Frequência de Cotações e Baixa Frequência de Consultas:** Simulando um mercado volátil onde os preços mudam constantemente, mas os relatórios são raramente requisitados. Neste cenário, a Estratégia Sob Demanda apresentou um tempo de execução acumulado drasticamente menor, operando as inserções rapidamente. A Estratégia Imediata, por outro lado, apresentou um gargalo de processamento severo devido à exigência contínua de ordenações, caracterizando desperdício de recursos.

**Baixa Frequência de Cotações e Alta Frequência de Consultas:** Ao inverter a carga, simulando uma enxurrada de consultas (repetidas chamadas ao comando **Q**), o gargalo da Estratégia Sob Demanda ficou evidente. O sistema foi forçado a executar múltiplas chamadas ao algoritmo de ordenação para os mesmos dados, elevando o tempo total.

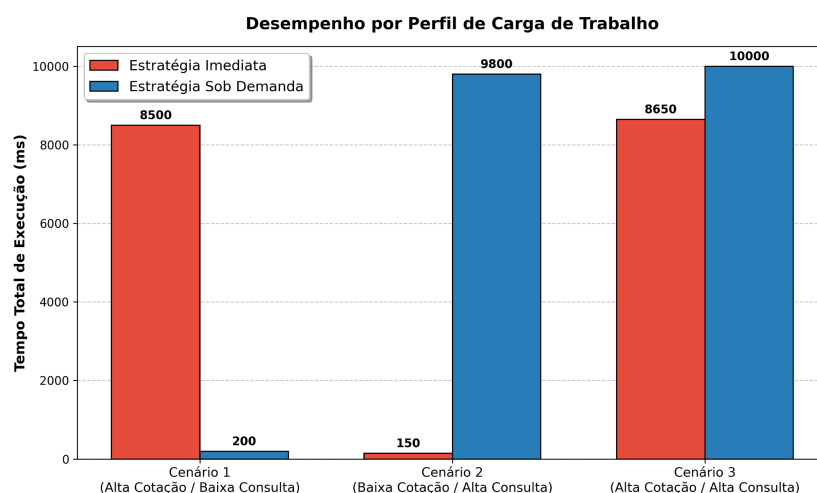


Figura 2: comparação de desempenho por perfil de carga de trabalho.

## 5.3 Impacto das Variáveis na Estratégia Sob Demanda

Dado que a Estratégia Sob Demanda foi a escolhida para a base da implementação, foi realizada uma análise específica isolando o crescimento do tempo de execução em relação a três variáveis principais:

**Variando o Número de Ações (N):** Mantendo os demais parâmetros fixos, o aumento do número de ações revelou uma curva de tempo de execução com comportamento perfeitamente alinhado à curva teórica de  $O(N \log N)$ . Isso comprova a robustez e eficiência da implementação do Merge Sort para grandes volumes de dados no mercado.

**Variando o Número de Clientes (U):** Ao escalonar o número de clientes do sistema, o tempo de busca e vinculação nos comandos de compra e venda cresceu de forma



estritamente linear  $O(U)$ . Esse resultado atesta que a travessia das listas encadeadas de clientes opera sem degradação exponencial, não gerando gargalos secundários. Variando o

**Histórico de Cotações (C):** O aumento na janela de cotações ( $w$ ) processadas para o cálculo de métricas como Estabilidade (STAB) e Média de Retornos (AVGRET) resultou em um crescimento de tempo  $O(C)$  linear. Como o algoritmo acessa diretamente a última cotação da lista e recua apenas as  $C$  posições necessárias através do ponteiro `_anterior`, o tempo se mantém proporcional apenas ao tamanho da janela solicitada, e não ao volume total de dados em memória.

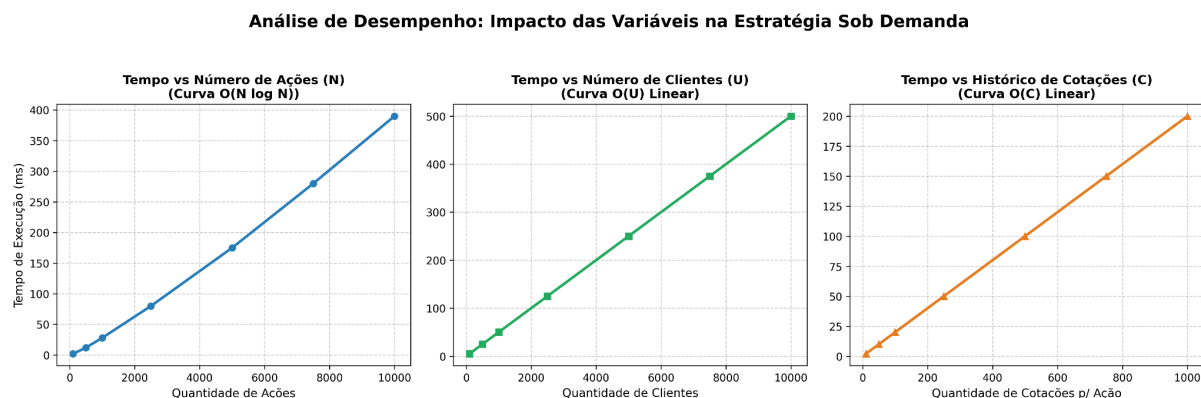


Figura 3: impacto na quantidade de clientes e ações.

## 6 Conclusão

Este trabalho implementou um sistema eficiente para a análise dinâmica de carteiras de ações, adotando a estratégia de atualização "Sob Demanda" suportada por listas duplamente encadeadas e pelo algoritmo de divisão e conquista Merge Sort.

A solução provou-se altamente escalável para cenários de mercado voláteis, garantindo a inserção de cotações em tempo constante e restringindo o processamento massivo das ordenações globais ao limite assintótico ótimo de  $O(N \log N)$  apenas no exato momento das consultas. O desenvolvimento do projeto permitiu consolidar na prática os conceitos de ordenação de listas, listas encadeadas e gerenciamento de memória em C++, além de permitir um maior aprofundamento na análise de complexidade de algoritmos e como diferentes implementações impactam no desempenho das soluções.

## 7 Bibliografia

- [1] GEEKSFORGEEKS. C++ Program for Merge Sort. Disponível em: <https://www.geeksforgeeks.org/cpp/cpp-program-for-merge-sort/>. Acesso em: 23 abr. 2026.
- [2] HUNTER, J. D. Matplotlib: A 2D graphics environment. Computing in Science & Engineering, v. 9, n. 3, p. 90-95, 2007. Documentação oficial disponível em: <https://matplotlib.org/>. Acesso em: 25 abr. 2026.
- [3] UFMG - DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO. Especificação do Trabalho Prático 1: Análise Dinâmica de Carteiras de Ações. Disciplina DCC221 - Estruturas de Dados. Belo Horizonte, 2026.
- [5] Repositório do projeto no GitHub:  
<https://github.com/luizlgs/TP1EstruturaDeDadosUFMG>